

AI FAQ Assistant API

4. Project Planning Phase

1. Introduction

This document outlines how the AI FAQ Assistant API project will be organized, scheduled, resourced, and managed from start to finish. It builds directly on the requirements and design already documented, translating them into a concrete plan of work, timeline, and risk strategy.

2. Project Scope

The scope covers designing, developing, and testing an AI-powered FAQ Assistant REST API with FAQ management, authentication, query logging, and basic feedback/analytics capabilities. The following items are explicitly considered out of scope for the first release and are noted as future enhancements:

- Multi-language question support
- Voice-based question input
- Fully generative (open-domain) answer generation without a curated FAQ base
- A polished, production-grade front-end application (a minimal demo interface is sufficient)

3. Project Objectives

- Deliver a working, documented REST API implementing all Must-have functional requirements
- Achieve acceptable answer accuracy on a representative test set of FAQ questions
- Complete the project within the planned schedule using the available team and tools
- Produce complete documentation covering each phase of the project lifecycle

4. Work Breakdown Structure (WBS)

Task	Sub-tasks
1. Requirement Analysis	Gather requirements; define functional/non-functional requirements; document scope
2. System Design	Define architecture; design database schema; design API contract
3. Environment Setup	Set up repository, server, database, and base project skeleton
4. Core Development	Build /api/ask endpoint; integrate NLP matching engine; implement FAQ CRUD
5. Admin & Authentication	Implement login, token issuance, and route protection for admin endpoints
6. Analytics & Feedback	Implement query logging, feedback capture, and basic reporting endpoints
7. Testing	Write and execute unit, integration, system, and user acceptance tests
8. Deployment & Documentation	Deploy to a server/cloud environment; finalize all project documentation

5. Project Timeline and Milestones

The project is planned over an approximate nine-week schedule, suitable for a small team or academic setting. Actual durations may be adjusted based on team size and availability.

Phase	Planned Duration	Key Deliverable
Brainstorming & Ideation	Week 1	Approved problem statement and concept document
Requirement Analysis	Week 2	Requirement Analysis document
System Design	Week 3	Architecture and database design document
Project Planning	Week 3 (parallel)	Project plan, WBS, schedule, and risk plan

Core Development	Weeks 4-6	Working /api/ask endpoint with NLP matching
Admin, Auth, Feedback Development	Week 7	FAQ management, authentication, feedback endpoints
Testing	Week 8	Test cases executed; defects logged and resolved
Deployment & Final Review	Week 9	Deployed API and completed documentation set

6. Resource Allocation and Team Roles

Role	Responsibility
Project Lead / Coordinator	Overall coordination, scheduling, and documentation quality
Backend Developer	API endpoint development, database integration, authentication
NLP / AI Engineer	Matching engine implementation, embedding generation, threshold tuning
QA / Tester	Test case design, execution, defect logging, and regression testing

On smaller teams, one person may hold more than one role; the responsibilities above remain the same regardless of team size.

7. Tools and Technologies Planned

Category	Tool / Technology
Backend framework	Node.js (Express) or Python (FastAPI/Flask)
Database	MySQL or PostgreSQL
NLP / matching	Sentence-embedding model or a third-party AI API for semantic similarity
Authentication	JWT-based token authentication
API documentation	Swagger / OpenAPI

Version control	Git and GitHub
API testing	Postman
Project tracking	Trello, Jira, or a shared task spreadsheet

8. Budget Considerations

Where possible, the project favors open-source and free-tier tools to keep costs minimal, which is appropriate for a pilot, academic, or small-team context:

- Open-source NLP libraries instead of paid, high-volume AI services where feasible
- Free-tier or low-cost cloud hosting for the pilot deployment
- Open-source database engines (MySQL/PostgreSQL) rather than commercial licenses

9. Risk Management Plan

Risk	Likelihood	Impact	Mitigation
Low match accuracy from the NLP engine	Medium	High	Tune confidence threshold; expand FAQ dataset; review low-confidence logs regularly
Scope creep beyond planned features	Medium	Medium	Freeze the Must-have feature list before development begins; treat extras as future scope
Delay in NLP component integration	Medium	High	Start NLP integration early, in parallel with API scaffolding, rather than last
Data security gaps	Low	High	Implement authentication and input validation from the first development sprint
Team member unavailability	Low	Medium	Cross-train team members on more than one module where possible

10. Communication Plan

- Short weekly check-ins to review progress against the WBS and timeline
- Shared task board to track status of each task (To Do / In Progress / Done)
- Version-controlled repository as the single source of truth for code and documentation

11. Dependency Notes

Certain tasks depend on the completion of earlier ones and cannot start in parallel. These dependencies are important for keeping the schedule realistic:

Task	Depends On
Core Development	System Design (architecture and database schema finalized)
Admin & Authentication	Core Development (basic API structure in place)
Testing	Completion of all development sprints
Deployment	Successful completion of Testing phase

12. Quality Assurance Approach

- Code reviews performed before merging any feature branch into the main branch
- Test cases written alongside development rather than only at the end, where practical
- A dedicated Testing phase (Section 7 of the overall project) formally verifies all requirements before deployment

13. Change Management

Any request to change scope after the plan is finalized is reviewed against the project timeline and Must-have requirement list before being accepted. Minor clarifications may be absorbed within existing tasks, while larger additions are deferred to a future release to protect the project schedule.

14. Monitoring and Progress Tracking

- Task board reviewed at each check-in to compare actual vs. planned progress
- Milestones from Section 5 used as checkpoints to detect schedule slippage early
- Risk register (Section 9) reviewed periodically and updated as new risks are identified

15. Assumptions Made During Planning

- Team members are available on a consistent part-time or full-time basis throughout the nine-week schedule
- The NLP library or API selected during design integration works as documented without major compatibility issues
- A representative starter FAQ dataset can be prepared in time for the Development and Testing phases

16. Roles and Responsibility Matrix

Task Area	Project Lead	Backend Developer	NLP Engineer	QA/Tester
Requirement Analysis	Reviews	Contributes	Contributes	Reviews
System Design	Reviews	Leads	Contributes	Reviews
Core Development	Tracks	Leads	Leads	Observes
Testing	Tracks	Supports	Supports	Leads
Deployment	Leads	Supports	Supports	Verifies

17. Contingency Planning

In addition to the risk mitigations listed in Section 9, the plan sets aside informal schedule buffer at the end of the Development phase and before final deployment, so that unexpected delays in one task do not automatically push back the entire project deadline. If a major risk materializes despite mitigation, the first response is to protect the Must-have requirement list, deferring Should-have features rather than compromising core functionality.

18. Detailed Sprint-Level Planning

To make the Core Development milestone in Section 5 more actionable, the work was further broken down into short, focused sprints. This finer-grained plan gives the team clear weekly targets and makes it easy to spot slippage early rather than only discovering a delay at the end of a multi-week milestone.

Sprint	Duration	Planned Outcome
Sprint 1	Week 4	Project skeleton, database schema created, basic health-check endpoint working
Sprint 2	Week 5	POST /api/ask working end-to-end with the NLP matching engine integrated
Sprint 3	Week 6	FAQ CRUD endpoints complete with category support
Sprint 4	Week 7	Authentication, admin route protection, feedback and logging complete

Each sprint ends with a short internal review where the team demonstrates the working feature to the rest of the group. This keeps everyone aligned on progress and allows early feedback before moving to the next sprint, reducing the chance of discovering integration problems late in the project.

19. Definition of Done

To avoid ambiguity about when a task is truly complete, the team agreed on a simple 'Definition of Done' checklist applied to every task in the Work Breakdown Structure:

- The relevant code is written, committed, and merged into the main branch
- The feature has been manually tested against its related functional requirement
- Any new environment variables or configuration steps are documented
- No known critical defects remain open against the feature

Applying this checklist consistently across every task made the transition into the formal Testing phase smoother, since most obvious issues had already been caught during development rather than being discovered for the first time during system testing.

20. Conclusion

This plan provides a realistic roadmap for delivering the AI FAQ Assistant API, covering scope, schedule, resources, tools, budget, dependencies, quality assurance, contingency planning, sprint-level detail, and risk management. It will be used to track progress through the Development and Testing phases and to confirm that the project stays aligned with its original objectives.